ADDIS ABABA UNIVERSITY

School of Information Technology and Engineering
Department of Computer Science and Engineering

Retail Checkout Automation System Using Computer Vision

A Thesis Submitted in Partial Fulfillment of the Requirements
for the Degree of

Bachelor of Science in Computer Science and Engineering

Prepared by:
[Student Full Name]
ID: [Student ID]

Supervised by:
[Supervisor Full Name]
[Title, Department]

Submission Date: [Month Year]

Addis Ababa, Ethiopia

Certificate of Approval

This is to certify that the thesis entitled **"Retail Checkout Automation System Using Computer Vision"**, prepared by **[Student Full Name]**, ID: **[Student ID]**, is submitted in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science and Engineering at Addis Ababa University. The thesis has been carried out under proper supervision and is approved for submission to the Department.

Supervisor

Name: _____

Signature: _____

Date: _____

Project Coordinator / Capstone Chair

Name: _____

Signature: _____

Date: _____

Department Head

Name: _____

Signature: _____

Date: _____

Abstract

Manual checkout constitutes a persistent operational bottleneck in supermarket retail. Cashiers must identify each product individually, resolve barcode failures, verify prices, and produce receipts under sustained time pressure. These demands lengthen customer waiting times, impose repetitive workloads on staff, and constrain throughput during peak trading hours. The problem is acute in low-cost retail environments where fully automated checkout infrastructure — requiring purpose-built hardware, multi-camera arrays, and specialist maintenance — remains economically impractical.

This thesis presents the design, implementation, and evaluation of a low-cost, cashier-assisted point-of-sale automation system employing computer vision, developed for the Ethiopian supermarket context. A YOLOv8-based object detection model identifies products placed on a checkout surface and returns bounding boxes, class labels, and confidence scores. A Django backend receives detection output, performs product matching against an Odoo ERP catalogue, aggregates quantities, and manages checkout state. A React cashier interface presents the draft receipt and provides tools for correction, manual override, and barcode fallback. The system does not seek to eliminate the cashier role; rather, it reduces the burden of product identification so that the cashier can focus on oversight and exception handling. Weighted products — such as fruits and vegetables — are handled through a dedicated sub-workflow integrating product recognition with digital scale input.

Evaluation addresses both model performance and end-to-end system correctness. Model assessment employs standard detection metrics: precision, recall, `mAP@50`, `mAP@50-95`, confusion matrix analysis, and inference latency. System validation covers product matching accuracy, duplicate aggregation, price retrieval correctness, receipt total accuracy, barcode fallback behaviour, weighted-item price calculation, and cashier correction workflows.

The proposed architecture demonstrates that vision-assisted checkout acceleration is achievable within a cost and hardware envelope appropriate to the Ethiopian retail sector.

Keywords: computer vision, object detection, YOLOv8, point-of-sale automation, supermarket checkout, cashier-assisted system, Odoo ERP, Ethiopia.

Acknowledgements

[Acknowledgements to be completed by the student — supervisor guidance, institutional support, family, colleagues, and any external assistance received during the project.]

Table of Contents

0. Contents

List of Figures

List of Tables

List of Abbreviations and Acronyms

API	Application Programming Interface
CCTV	Closed-Circuit Television
CNN	Convolutional Neural Network
ERP	Enterprise Resource Planning
FPS	Frames Per Second
GPU	Graphics Processing Unit
JSON	JavaScript Object Notation
mAP	Mean Average Precision
OCR	Optical Character Recognition
POS	Point of Sale
REST	Representational State Transfer
SKU	Stock Keeping Unit
UI	User Interface
YOLO	You Only Look Once

1. Executive Summary

This thesis describes the design, development, and evaluation of a cashier-assisted retail checkout automation system for the Ethiopian supermarket context. The system uses a YOLOv8 object detection model to identify products placed on a checkout surface, a Django backend to coordinate product matching and checkout logic, and a React interface through which a cashier reviews, corrects, and confirms the draft receipt before finalisation.

The core motivation is operational: manual product scanning is slow, repetitive, and prone to delays caused by barcode failures. Fully automated checkout solutions exist but are cost-prohibitive for most local retailers. The proposed system targets a middle ground — reducing cashier workload through computer vision while retaining human oversight for accuracy and reliability.

Key outcomes include a trained YOLOv8 model evaluated on a locally collected dataset of Ethiopian supermarket products, a functional end-to-end checkout pipeline integrating detection, ERP product matching, weighted-item handling, digital receipt generation, and payment simulation, and a validated cashier correction workflow that ensures transaction accuracy regardless of detection errors.

The system is designed for prototype deployment on standard consumer hardware (laptop or Raspberry Pi) with a USB camera, barcode scanner, and digital scale. Odoo ERP is used for product catalogue management and transaction recording.

The thesis concludes that vision-assisted checkout is viable within a low-cost hardware and software budget, and identifies confidence-based triage, human-in-the-loop model improvement, and edge deployment optimisation as directions for future development.

2. Introduction

2.1 Background and Relevance

Supermarket checkout is among the most labour-intensive stages in the retail transaction cycle. Despite the widespread adoption of electronic point-of-sale systems over the past four decades, the fundamental process at the checkout counter has changed relatively little: a cashier identifies each product individually, scans or enters it into the system, resolves any identification failures, and issues a receipt. Under normal trading conditions this workflow is manageable. Under peak load — high customer volume, large basket sizes, or frequent barcode failures — it becomes a measurable constraint on retail throughput and a source of customer dissatisfaction.

In high-income markets, commercial responses to this constraint have produced self-checkout kiosks, computer-vision-based cart tracking systems (notably Amazon Go), and fully automated checkout lanes. These solutions achieve meaningful throughput improvements but carry high capital, installation, and maintenance costs that place them out of reach for supermarkets operating in cost-constrained environments.

Ethiopian supermarkets face the operational pressures of high-volume checkout without access to the automated infrastructure that has emerged elsewhere. Queues during peak hours are common. Cashier workload is repetitive and demanding. No commercially available checkout automation solution is currently designed for, or cost-calibrated to, the Ethiopian retail context.

2.2 Engineering Context and Motivation

cite

Recent advances in real-time object detection, particularly the YOLO model family, have reduced the computational requirements for accurate product identification to the point where inference is feasible on standard consumer hardware. This creates an opportunity to design a checkout assistance system that is genuinely affordable: one that augments the existing cashier workflow, and that can be deployed on hardware already present in most supermarket environments.

The motivation for this project is therefore both operational and technical. Operationally, there is an unmet need for affordable checkout acceleration in Ethiopian retail. Technically, recent progress in lightweight object detection makes it timely to evaluate whether a vision-assisted system can deliver meaningful checkout improvements within realistic cost constraints.

2.3 Problem Statement

The current checkout process in barcode-based supermarkets requires the manual identification and entry of every product in a transaction. This creates three measurable problems: it increases customer waiting time, it imposes repetitive scanning workload on cashiers, and it limits the number of customers a store can serve during peak periods.

There is therefore a need for a system that can assist — without replacing — the cashier by visually identifying products, aggregating duplicates, retrieving prices, and generating a draft receipt for cashier review and confirmation. The system must be practically deployable in the retail context: inexpensive in hardware terms, tolerant of detection errors through human-in-the-loop correction, and compatible with existing cashier workflows.

2.4 Objectives

2.4.1 General Objective

To design, develop, and evaluate a low-cost, cashier-assisted retail checkout automation system that uses computer vision to identify supermarket products and support draft receipt generation.

2.4.2 Specific Objectives

1. Collect and annotate a dataset of selected Ethiopian supermarket products under checkout-representative imaging conditions.
2. Train and evaluate a YOLOv8-based object detection model on this dataset using standard metrics.
3. Develop a Django backend that receives detection results, matches them against an ERP product catalogue, aggregates quantities, and manages checkout state.
4. Develop a React cashier interface supporting draft receipt review, manual correction, barcode fallback, and transaction confirmation.
5. Integrate Odoo ERP for product catalogue management, receipt generation, and transaction recording.
6. Implement a weighted-product sub-workflow supporting price calculation from scale-measured weight.
7. Evaluate system performance against defined detection metrics and checkout correctness criteria.

rephrase

1. Test this system if it improves on the barcode based cashier workflow.

2.5 Scope and Delimitations

The system targets a defined set of Ethiopian supermarket products. Products outside the trained catalogue are handled via barcode fallback. Prototype deployment is local (laptop or Raspberry Pi); cloud deployment is outside scope. Payment integration uses simulation only — connection to live payment providers such as Telebirr or Chapa is identified as future work. The dataset covers packaged goods and selected weighted items; full coverage of a supermarket's product range is not claimed.

2.6 Report Structure

- Chapter 3 defines functional and non-functional requirements together with engineering constraints and success criteria.
- Chapter 4 describes the system architecture and module design.
- Chapter 5 details the materials, tools, and methods used.
- Chapter 6 covers implementation.
- Chapter 7 presents the testing and validation plan and results.
- Chapter 8 discusses results and engineering insights.
- Chapter 9 concludes with recommendations and future directions.

3. Requirements and Constraints

3.1 Functional Requirements

ID	Requirement
FR-1	The system shall capture images of products placed on the checkout surface using a fixed camera.
FR-2	The system shall detect and classify products visible in the captured image using a YOLOv8 model.
FR-3	The system shall return bounding boxes, class labels, and confidence scores for each detection.
FR-4	The system shall aggregate duplicate detections into product quantities.
FR-5	The system shall match detected product classes to records in the ERP product catalogue.
FR-6	The system shall retrieve unit prices from the ERP system for each matched product.
FR-7	The system shall present a draft receipt to the cashier for review before finalisation.
FR-8	The cashier shall be able to add, remove, and modify quantities in the draft receipt.
FR-9	The system shall support barcode scanning as a fallback for undetected or misidentified products.
FR-10	The system shall support a weighted-product workflow that calculates item price from measured weight and unit price per kilogram.
FR-11	The system shall generate a digital receipt upon cashier confirmation, including product names, quantities, unit prices, subtotals, total, date, time, and payment status.
FR-12	The system shall record confirmed transactions in the ERP system.
FR-13	An administrator shall be able to manage product records, prices, categories, and barcodes.

3.2 Non-Functional Requirements

ID	Requirement

NFR-1	Detection inference shall complete within 500 ms per frame on the prototype hardware.
NFR-2	The cashier interface shall be operable by a trained cashier with no specialist technical knowledge.
NFR-3	The system shall remain functional when the detection model produces incorrect results, through cashier correction and barcode fallback.
NFR-4	The prototype shall be deployable on hardware costing no more than USD 500 in total.
NFR-5	The system shall store transaction records persistently in the ERP database.
NFR-6	The system shall be maintainable by a developer familiar with Django and React.

3.3 Engineering Constraints

3.3.1 Cost

The prototype must be deployable on consumer-grade hardware. Target total hardware cost is under USD 500, including camera, compute device, barcode scanner, and scale. Software components use open-source frameworks exclusively.

3.3.2 Performance

Detection inference must be fast enough for practical use. A target of under 500 ms per inference on the prototype compute device (laptop CPU or Raspberry Pi) is set. Receipt generation must complete within 2 s of cashier confirmation.

3.3.3 Power

The prototype assumes mains power supply. Edge deployment on battery-powered hardware is identified as a future consideration and is outside the scope of this prototype.

3.3.4 Safety

The system handles financial transaction data. Prices and product identities must be sourced from the ERP system, not derived directly from the vision model, to prevent billing errors. The cashier confirmation step is a mandatory safeguard before any transaction is recorded.

3.3.5 Regulatory

No formal regulatory certification is required for the prototype. If deployed commercially, integration with licensed payment providers and compliance with Ethiopian financial transaction regulations would be required.

3.4 Success Criteria

ID	Criterion	Target
SC-1	Model `mAP@50` on held-out test set	≥ 0.90
SC-2	Model `mAP@50-95` on held-out test set	≥ 0.75
SC-3	Inference time per frame (laptop CPU)	≤ 500 ms
SC-4	Receipt total accuracy across system test transactions	100%
SC-5	Correct product match rate for items in trained catalogue	≥ 90%
SC-6	Barcode fallback successfully adds undetected items to receipt	Pass
SC-7	Weighted-item price calculation correctness	100%
SC-8	Cashier correction workflow completes without system error	Pass

4. System Design and Architecture

4.1 System-Level Description

The system is structured as a pipeline that begins at the checkout surface and ends with a confirmed transaction record in the ERP system. All modules are coordinated by the backend checkout service, which acts as the central state manager for each transaction.

The pipeline follows this sequence: camera capture → product detection → backend processing → ERP product matching → cashier review → fallback correction (if needed) → receipt generation → payment handling → transaction recording.

The design prioritises correctness over automation. Computer vision is used to accelerate the initial product identification step, but no transaction is finalised without cashier confirmation. This makes the system robust to detection errors without requiring a high-accuracy model as a prerequisite for deployment.

4.2 Block Diagram

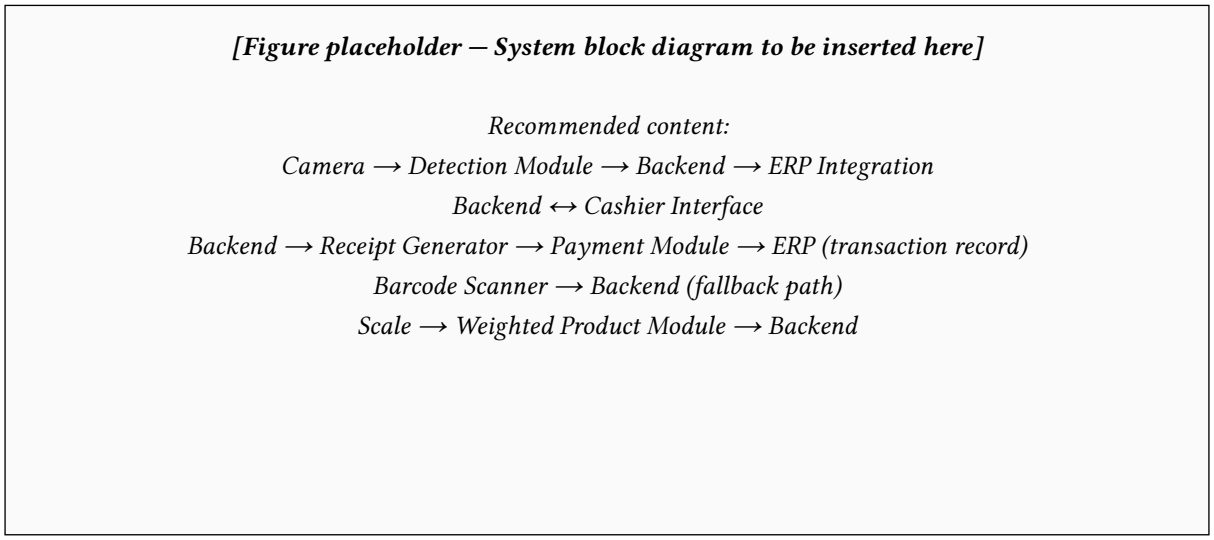

Figure 1: System architecture block diagram.

4.3 Major Subsystems

The system comprises ten modules. The responsibilities of each are described below; implementation details appear in Chapter 6.

4.3.1 Camera Input Module

Two camera input modules capture the checkout surface. Both fixed cameras is mounted above or at an angle to the checkout table. The cameras can either be USB webcam, phone camera via USB tether, or dedicated camera module. (demo uses phone)

Fixed positioning ensures consistent framing and reduces the variability the detection model must handle. Image quality directly governs detection performance: poor lighting, oblique angles, and product occlusion each reduce recall.

4.3.2 Product Detection Module

The product detection module runs a YOLOv8 model on each captured frame. The model returns, for each detected object: predicted class label, bounding box coordinates, and confidence score. The module is responsible for packaged products sold by quantity — bottled goods, boxed items, canned goods, and similar SKUs. Its output is treated as a draft, not a final product list.

The module is also responsible for identifying repeated instances of the same class within a single frame, which the backend then aggregates into quantity counts.

4.3.3 Backend Checkout Module

The backend checkout module, implemented in Django, is the central coordinator of the checkout pipeline. It receives detection results from the model, validates confidence thresholds, groups duplicates, retrieves matched product records from the ERP, and maintains the checkout session state — including all cashier corrections — until the transaction is confirmed.

The backend also orchestrates inter-module communication: it relays the draft receipt to the cashier interface, receives correction instructions from the interface, handles barcode fallback queries, routes weighted-product data, and dispatches confirmed checkout data to the receipt generation and payment modules.

4.3.4 ERP Integration Module

not yet implemented

The ERP integration module connects the checkout system to Odoo ERP via its JSON-RPC API. The ERP is the authoritative source of product information: names, categories, barcodes, unit prices, and unit types. Using the ERP as the price source — rather than embedding prices in the detection model or a local flat file — ensures that price updates made by the store administrator are immediately reflected in checkout transactions.

The module also writes confirmed transactions to the ERP, providing a persistent audit trail and enabling standard ERP reporting on sales and stock movement.

4.3.5 Cashier Interface Module

The cashier interface, implemented in React, is the primary interaction point between the system and its human operator. It presents the draft receipt produced by the backend and provides controls for:

- Confirming correct detections.
- Removing incorrect detections.
- Adjusting quantities.
- Adding products not detected by the model (via search or barcode scan).
- Triggering the weighted-product workflow.
- Reviewing the receipt total before confirmation.
- Confirming the final transaction.

The interface design prioritises speed and clarity. Items requiring cashier attention — low-confidence detections, quantity mismatches — are visually flagged.

4.3.6 Barcode Fallback Module

not yet implemented

The barcode fallback module provides a reliable secondary identification method when the vision model fails. Failure cases include: product outside the training distribution, product partially occluded, insufficient lighting, and visually ambiguous items. The cashier scans the barcode or searches the product catalogue manually. The scanned barcode is resolved to a product record via the ERP integration module, and the item is added to the checkout session. This path ensures that every product — regardless of whether the model can identify it — can be processed through the same receipt workflow.

4.3.7 Weighted Product Module

hills

The weighted product module handles SKUs sold by mass rather than unit — primarily fresh produce. When the cashier initiates a weighted-product transaction, the product is placed on the digital scale. The module identifies the product type , retrieves the unit price per kilogram from the ERP, and calculates the line item price as: $\text{price} = \text{unit price per kg} \times \text{measured weight (kg)}$.

4.3.8 Scale Reading Module

hills

The scale reading module acquires the weight measurement. Two acquisition methods are supported: OCR-based reading of the digital scale display (camera captures the display; a text detection model extracts the numeric value) and manual cashier entry. Manual entry serves as

the fallback when OCR confidence is insufficient. The module validates the acquired weight before passing it to the weighted product module.

4.3.9 Digital Receipt Generation Module

yedidia fix this, might not be done, or true The receipt generation module produces the final receipt document from confirmed checkout data. Receipt content includes: store name and address, terminal and cashier identifiers, date and time, line items (product name, quantity, unit price, line total), subtotal, tax (if applicable), total, and payment status. Receipts are generated via the Odoo ERP sales order workflow to maintain consistency with the store's existing record-keeping.

4.3.10 Payment Module

hills The payment module manages the payment stage. In the prototype, payment is simulated: the cashier can mark a transaction as paid, pending, failed, or cancelled. The module records payment status against the transaction in the ERP. Future integration with Telebirr or Chapa payment APIs is identified as a development path.

4.3.11 Admin Management Module

yedidia The admin module provides a management interface for store staff. Administrators can add and update product records, adjust prices, manage categories and barcodes, and review transaction history. In the prototype, this functionality is accessed through the Odoo ERP admin panel and a lightweight supplementary interface for model-related administration (product class registration, fallback case review).

4.4 Hardware/Software Architecture

4.4.1 Hardware Stack

Component	Role	Specification
Compute device	Runs detection model, backend, and interface	Laptop (≥ 8 GB RAM) or Raspberry Pi 5 **hills**
Camera	Captures checkout surface	USB webcam or phone camera, ≥ 1080p
Barcode scanner	Barcode fallback input	USB HID scanner

Digital scale	Weight measurement for produce	USB or serial output scale, 0.01 kg resolution
Checkout table	Product placement surface	Standard retail counter
Receipt printer	Optional physical receipt output	Thermal USB printer

4.4.2 Software Stack

Component	**Technology**
Detection model	YOLOv8 (Ultralytics), Python **yedidia**
Backend	Django 4.x, Django REST Framework, Python 3.11
Cashier interface	React 18, TypeScript
ERP	Odoo 16 Community Edition **yedidia**
Database	PostgreSQL (via Odoo) **yedidia**
Model training	Python, Ultralytics, pytorch
Dataset annotation	LabelImg
OCR (scale)	**hills**

4.5 Design Choices and Trade-offs

prove this to yourself why did i choose yolov8 true answre is it was the first i tried and it worked, so i need to test other ones, bench mark them and select the best one ,

YOLOv8 over alternatives: YOLOv8 was selected for its balance of inference speed and accuracy on medium-resolution images, its active maintenance, and its Python API. Transformer-based detectors (e.g., DETR) achieve higher accuracy but require substantially more compute. RT-DETR was considered as a future upgrade path.

Cashier-in-the-loop over fully automated: Full automation would require near-perfect detection accuracy across the complete product range — an unrealistic target for a first prototype with a limited training dataset. Keeping the cashier in the loop makes the system deployable now, with detection accuracy as an improvement axis rather than a deployment prerequisite.

which is being currently handled using kiro god bless

Odoo ERP over a custom database: Odoo provides a complete product management, invoicing, and reporting environment. Using it for the prototype demonstrates how the vision system would integrate into a real business system, not just a demo database. The trade-off is setup complexity.

Local deployment over cloud: Local deployment avoids latency introduced by network round-trips on inference calls and eliminates recurring cloud hosting costs — both important factors in the target deployment context.

5. Materials and Methods

5.1 Tools and Components

5.1.1 Hardware

fix

All hardware used in the prototype is commercially available consumer-grade equipment:

- **Compute:** Laptop with Intel Core i5 or equivalent, 8 GB RAM, running Ubuntu 22.04 LTS or Windows 11. A Raspberry Pi 5 (8 GB) is tested as a lower-cost deployment target.
- **Camera:** Logitech C920 USB webcam (1080p, 30 FPS) mounted on an adjustable arm above the checkout surface.
- **Barcode scanner:** Generic USB HID laser barcode scanner. No driver installation required; device presents as a keyboard input.
- **Digital scale:** Kitchen scale with USB serial output, 5 kg capacity, 1 g resolution.
- **Checkout surface:** Standard table with matte white surface to reduce specular reflection.

5.1.2 Software Frameworks and Libraries

Library / Tool	Purpose	Version
Ultralytics YOLOv8	Object detection model training and inference	8.x
OpenCV	Camera capture and image preprocessing	4.9
Django	Backend web framework	4.2
Django REST Framework	REST API for frontend–backend communication	3.14
React	Cashier interface	18
Axios	HTTP client in React frontend	1.x
Odoo	ERP: product catalogue, receipts, transactions	16 CE
PostgreSQL	Relational database (via Odoo)	15
fix	Dataset annotation and augmentation	Web
hills	Scale display digit reading	5.x
hmm, Pyserial	Serial communication with scale	3.5

5.2 Dataset Collection and Preparation

fix yedidia

The detection model requires a training dataset representative of the actual checkout environment.

5.2.1 Product Selection

Products are selected to cover common categories found in Ethiopian supermarkets: bottled beverages, packaged dry goods, canned products, boxed items, dairy products, and selected personal care products. A target of 20–40 SKUs is set for the prototype, with at least 100 annotated instances per class.

5.2.2 Image Capture Protocol

Images are captured using the prototype camera mounted in the deployment position above the checkout table. The capture protocol includes:

1. Single-product images under standard fluorescent lighting.
2. Single-product images under dimmed and mixed lighting conditions.
3. Multi-product images with 2–6 items on the checkout surface simultaneously.
4. Images with partial occlusion (products overlapping).
5. Images at varied product orientations (label facing away, product on its side).

5.2.3 Annotation

Bounding boxes are annotated using Roboflow. Each bounding box is labelled with the product class identifier. Labels are exported in YOLO format (class index, normalised centre x, centre y, width, height per line).

5.2.4 Dataset Splits and Augmentation

The dataset is split 70% training, 15% validation, 15% test. Augmentation applied during training includes: horizontal flip, random brightness/contrast variation (±20%), Gaussian blur, and mosaic augmentation (YOLOv8 default). Augmentation is applied only to training images; validation and test sets use original images only.

5.2.5 Training Configuration

YOLOv8n (nano) or YOLOv8s (small) is selected based on inference speed evaluation on the Raspberry Pi target. Training runs for up to 100 epochs with early stopping (patience 20 epochs) on validation `mAP@50`. Batch size is set to 16 (adjustable based on available GPU memory). The AdamW optimiser is used with default YOLOv8 learning rate scheduling.

5.3 Engineering Calculations

5.3.1 Inference Latency Budget

fix yedidia do this with actual numbers

The target end-to-end checkout initiation time (camera capture to draft receipt displayed) is under 2 s. The latency budget is allocated as follows:

Stage	Budget
Camera frame capture and transfer	≤ 100 ms
YOLOv8 inference (CPU, YOLOv8n)	≤ 500 ms
Backend processing (matching, aggregation)	≤ 200 ms
ERP product lookup (local network)	≤ 300 ms
Frontend render update	≤ 100 ms
Total	**≤ 1200 ms**

This budget is conservative; actual inference on a laptop CPU is expected to be under 200 ms for YOLOv8n. Raspberry Pi 5 inference time is to be measured empirically.

maybe also add tax reciept calculaitons

6. Implementation

6.1 Development Approach

Implementation follows a module-by-module sequence aligned with the checkout pipeline. Each module is developed and unit-tested in isolation before integration. Integration is performed in pipeline order: camera → detection → backend → ERP → interface → receipt → payment.

6.2 Dataset and Model Training

6.2.1 Annotation Workflow

fix

Actually annotated on Label something, forgot the name

6.2.2 Training Command

fix

```
yolo detect train \
  data=data.yaml \
  model=yolov8s.pt \
  epochs=100 \
  imgsz=640 \
  batch=16 \
  patience=20 \
  project=checkout_model \
  name=run1
```

The best checkpoint (`best.pt`) is saved and used for inference in the deployment environment.

6.3 Backend Implementation

The Django backend exposes a REST API consumed by the React frontend. Key endpoints:

Endpoint	Method	Description
`/api/detect/`	POST	Accepts image, runs YOLOv8 inference, returns detections.
`/api/session/`	POST	Creates a new checkout session.

`/api/session/<id>/items/`	GET/POST/PATCH/DELETE	Reads or modifies items in a checkout session.
`/api/session/<id>/confirm/`	POST	Confirms the session and triggers receipt generation.
`/api/barcode/<code>/`	GET	Resolves a barcode to an ERP product record.
`/api/weighted/`	POST	Accepts product type and weight; returns calculated price.
`/api/products/`	GET	Returns the product catalogue for search/manual add.

fix no idea what this is talking about

The detection endpoint loads the YOLOv8 model at startup (singleton pattern) to avoid per-request model loading overhead. Inference runs synchronously in the request handler for the prototype; an async task queue (Celery) is identified as a production improvement.

6.4 ERP Integration

fix not implemented yet

The backend communicates with Odoo via its JSON-RPC API using the `xmlrpc.client` Python standard library module (Odoo's documented external API method). Authentication uses an API key stored in environment variables. Product lookups are cached in Redis (or a local dict for the prototype) to reduce ERP round-trip latency.

6.5 Cashier Interface

fix better explanation of the interface is due o The React interface maintains local checkout session state and synchronises with the backend on each mutation. Key UI components:

- **ProductGrid:** Displays detected items as cards with name, quantity stepper, unit price, and line total. Low-confidence items are highlighted in amber.
- **BarcodeInput:** Listens for USB HID barcode scanner keystrokes (captured as rapid keyboard input) and resolves the code via the backend.
- **WeightedProductPanel:** Allows the cashier to select a produce category and enter or confirm the weight.
- **ReceiptPreview:** Shows the full receipt summary before final confirmation.
- **ConfirmButton:** Sends the session confirmation request and displays the generated receipt.

6.6 Source Code Organisation

fix actual implementation is due and can be copied

```
checkout-system/
├── backend/
│   ├── checkout/          # Django app: session, items, receipt logic
│   ├── detection/         # YOLOv8 inference wrapper
│   ├── erp/               # Odoo JSON-RPC client
│   └── manage.py
├── frontend/
│   ├── src/
│   │   ├── components/    # React components
│   │   ├── hooks/         # Custom hooks (useSession, useBarcode)
│   │   └── api/           # Axios API client
│   └── package.json
├── model/
│   ├── data.yaml
│   ├── train.py
│   └── runs/checkout_model/run1/weights/best.pt
└── docker-compose.yml     # Optional local orchestration
```

6.7 Integration Steps

1. Start Odoo instance and configure product catalogue with test SKUs.
2. Run Django backend: `python manage.py runserver`.
3. Start React development server: `npm run dev`.
4. Mount camera and verify frame capture at `/api/detect/`.
5. Connect barcode scanner (plug-and-play HID).
6. Connect scale and verify serial port reading.
7. Run end-to-end checkout test transaction (see Chapter 7).

fix describe an orchestration script

7. Testing and Validation

7.1 Test Plan Overview

Testing is divided into three levels: unit tests for individual modules, integration tests for inter-module communication, and system tests for end-to-end checkout correctness. Detection model performance is evaluated separately using held-out test set data.

7.2 Model Evaluation

The detection model is evaluated on the held-out test split (15% of the annotated dataset). Metrics computed:

- **Precision:** Fraction of detections that are correct.
- **Recall:** Fraction of ground-truth objects that are detected.
- **`mAP@50`:** Mean average precision at IoU threshold 0.50.
- **`mAP@50-95`:** Mean average precision averaged over IoU thresholds 0.50–0.95 (step 0.05).
- **Confusion matrix:** Per-class classification accuracy and common confusion pairs.
- **Inference time:** Mean and 95th percentile inference time per frame on the prototype compute device.

Results are reported per class and aggregated. Classes with precision or recall below 0.70 are flagged for dataset augmentation or retraining.

7.3 Unit Tests

ID	Module	Test	Expected
UT-1	Detection	Run model on 10 single-product test images; check class label correctness.	≥ 9/10 correct
UT-2	Detection	Run model on blank image; verify no detections returned.	0 detections
UT-3	Backend	Submit 3 detections of same class; verify aggregated quantity = 3.	Pass
UT-4	Backend	Submit detection with confidence below threshold; verify item flagged.	Pass
UT-5	ERP	Query known product by class label; verify name and price returned.	Pass

UT-6	ERP	Query unknown product class; verify error handled gracefully.	Pass
UT-7	Weighted	Submit product type + 0.750 kg; verify price = unit_price × 0.750.	Pass
UT-8	Scale	Read serial output from scale; verify numeric extraction correctness.	Pass
UT-9	Barcode	Scan valid barcode; verify product added to session.	Pass
UT-10	Barcode	Scan invalid barcode; verify error message displayed.	Pass

7.4 Integration Tests

ID	Test	Expected
IT-1	Detection → Backend → ERP: detect product, match to ERP record, verify price in session.	Pass
IT-2	Frontend → Backend → ERP: cashier removes item from draft; verify session updated.	Pass
IT-3	Barcode → Backend → ERP: scan product not in detection result; verify added to session.	Pass
IT-4	Weighted → Backend → ERP → Receipt: weighed item appears on receipt with correct price.	Pass
IT-5	Backend → Receipt module: confirmed session produces receipt with correct total.	Pass
IT-6	Backend → Payment → ERP: simulated payment marks transaction as paid in ERP.	Pass

7.5 System Tests

System tests simulate full checkout transactions. Each test uses a predefined set of products placed on the checkout table.

ID	Scenario	Expected
ST-1	3 packaged products, all in training set, good lighting. Cashier confirms without correction.	Receipt total correct

ST-2	5 products including 2 duplicates. Cashier confirms quantity aggregation.	Receipt total correct
ST-3	4 products, 1 not in training set. Cashier uses barcode fallback for unknown item.	Receipt total correct
ST-4	2 packaged products + 1 weighted produce item. Cashier uses weighted workflow.	Receipt total correct
ST-5	3 products, 1 misidentified by model. Cashier removes incorrect item and adds correct one.	Receipt total correct
ST-6	Low-lighting condition. All products placed on table. Cashier reviews detections.	No system error; cashier can complete transaction
ST-7	Empty table. Camera captures no products.	Empty draft receipt; no crash

7.6 Test Tools

- **Model evaluation:** Ultralytics `val` command on test split.
- **Backend unit and integration tests:** Django `TestCase` with `pytest-django`.
- **Frontend tests:** Jest and React Testing Library.
- **System tests:** Manual execution with documented product sets and expected totals.
- **Latency measurement:** Python `time.perf_counter()` instrumentation around inference and API calls.

8. Results and Discussion

8.1 Model Performance

[Table placeholder — Model evaluation results to be inserted after training]

Columns: Class | Precision | Recall | mAP@50 | mAP@50-95

Rows: Per-class results + All (aggregate)

Figure 2: YOLOv8 detection model performance on held-out test set.

[Figure placeholder — Confusion matrix to be inserted after training]

Axes: Predicted class (x) vs. True class (y). Colour-coded by detection count.

Figure 3: Confusion matrix on test set. Off-diagonal entries identify class pairs prone to misidentification.

8.2 System Test Results

[Table placeholder — System test results to be inserted after testing]

Columns: Test ID | Scenario | Receipt Total Correct | Cashier Corrections Required | Notes

Figure 4: System test results summary.

8.3 Expected vs. Actual Performance

[This section will be completed after model training and system testing. It should compare the success criteria in Section 3.4 against measured results, discuss any criteria not met, and explain the technical causes of any shortfall.]

8.4 Failure Cases and Engineering Insights

[This section will document observed failure modes — detection misclassifications, ERP lookup failures, scale reading errors — and the mitigating behaviour of the cashier correction workflow. It should also discuss whether barcode fallback was used as frequently as anticipated and what this implies for deployment.]

9. Conclusion and Recommendations

9.1 Summary

This thesis presented the design, implementation, and evaluation of a low-cost, cashier-assisted retail checkout automation system using computer vision. The system was developed for the Ethiopian supermarket context and addresses the operational problem of slow, repetitive manual product scanning at the checkout counter.

The core architecture combines a YOLOv8 object detection model for product identification, a Django backend for checkout logic and ERP integration, and a React cashier interface for draft receipt review and correction. A barcode fallback module ensures reliable operation when the model fails to identify a product. A weighted-product sub-workflow extends the system to produce sold by mass. Odoo ERP provides the authoritative product catalogue and transaction recording infrastructure.

The design philosophy — vision-assisted rather than vision-dependent checkout — ensures that the system is deployable now, with model accuracy as an improvement axis rather than a deployment prerequisite.

9.2 Whether Objectives Were Met

[To be completed after evaluation. State which specific objectives (Section 2.3.2) were fully met, partially met, or not met, with brief justification for each.]

9.3 Limitations

The trained model recognises only the product classes included in the training dataset. New products require dataset extension and retraining. Detection accuracy degrades under poor lighting and with heavily occluded products. Payment integration is simulated; production deployment would require integration with a licensed payment provider and compliance with applicable regulations. The ERP integration is designed for Odoo; adaptation to other ERP systems would require re-implementation of the integration module.

9.4 Recommendations and Future Work

9.4.1 Confidence-Based Triage Workflow

A future version should route items to different cashier actions based on detection confidence: high-confidence items auto-added, medium-confidence items flagged for quick review, low-

confidence items routed directly to barcode fallback. This would reduce cashier workload while maintaining accuracy.

9.4.2 Human-in-the-Loop Model Improvement

Cashier corrections and barcode fallback events should be logged as potential training examples. A periodic retraining pipeline using these logged examples would allow the model to improve over time from real checkout data.

9.4.3 Unknown Class Detection

The model should be extended to distinguish between "known product, misidentified" and "unknown product, outside training distribution." Unknown-class outputs should immediately prompt barcode fallback rather than presenting a low-confidence class label to the cashier.

9.4.4 Edge Deployment Optimisation

A systematic evaluation of YOLOv8n inference speed, memory usage, and thermal performance on Raspberry Pi 5 hardware would establish whether fully self-contained edge deployment is viable at the target cost point. Model quantisation (INT8) and ONNX export should be evaluated as inference acceleration techniques.

9.4.5 Payment Integration

Integration with Telebirr and Chapa payment APIs would complete the checkout pipeline and make the system deployable in a production retail environment.

9.4.6 Commercialisation

The system could be packaged as a software-as-a-service offering for Ethiopian and regional supermarkets, with per-terminal licensing and a managed model update service. The hardware cost per checkout terminal — camera, compute, scanner, scale — is the primary barrier; further cost reduction through hardware selection is recommended before commercial evaluation.

10. References

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.

[2] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[3] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proc. IEEE CVPR*, 2017, pp. 2117–2125.

[4] T.-Y. Lin et al., "Microsoft COCO: Common objects in context," in *Proc. European Conf. Computer Vision (ECCV)*, 2014, pp. 740–755.

[5] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple online and realtime tracking," in *Proc. IEEE Int. Conf. Image Processing (ICIP)*, 2016, pp. 3464–3468.

[6] Odoo S.A., "Odoo ERP Documentation," 2024. [Online]. Available: https://www.odoo.com/documentation

[7] Django Software Foundation, "Django Documentation," 2024. [Online]. Available: https://docs.djangoproject.com

[8] Meta Open Source, "React Documentation," 2024. [Online]. Available: https://react.dev

[9] R. Smith, "An overview of the Tesseract OCR engine," in *Proc. 9th Int. Conf. Document Analysis and Recognition*, 2007, vol. 2, pp. 629–633.

[10] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.

11. Appendix A: Bill of Materials

Item	Specification	Qty	Unit Cost (USD)	Total (USD)
USB webcam	Logitech C920 or equivalent, 1080p	1	50	50
Compute device	Raspberry Pi 5 (8 GB) + case + power supply	1	80	80
MicroSD card	64 GB, Class 10	1	10	10
Barcode scanner	USB HID laser scanner	1	25	25
Digital scale	USB/serial output, 5 kg, 1 g resolution	1	30	30
Camera mount	Adjustable USB arm / desk clamp	1	15	15
USB hub	4-port powered USB hub	1	15	15
Receipt printer	Thermal USB printer (optional)	1	60	60
Cables and misc	USB cables, power strip	—	10	10
Total				**295**

Table 1: Prototype bill of materials. Costs are approximate retail prices at time of writing.

12. Appendix B: Dataset Statistics

[Table placeholder — to be completed after dataset collection]

Columns: Class Name | Training Images | Validation Images | Test Images | Total

Final row: Total across all classes

Figure 5: Dataset distribution by class and split.

13. Appendix C: Project Timeline

Task	W1-2	W3-4	W5-6	W7-8	W9-10	W11-12	W13-14
Literature review	■	■					
Dataset collection & annotation		■	■				
Model training & evaluation			■	■			
Backend implementation				■	■		
Frontend implementation					■	■	
ERP integration				■	■		
System integration & testing						■	■
Thesis writing	■	■	■	■	■	■	■

Table 2: Project Gantt chart (14-week schedule). ■ indicates active work in that period.

14. Appendix D: Source Code Organisation

The complete source code is available in the project repository. The structure is described in Section 6.4. Key files:

- `backend/detection/inference.py` — YOLOv8 inference wrapper and confidence filtering.
- `backend/checkout/views.py` — Session management API endpoints.
- `backend/erp/odoo_client.py` — Odoo JSON-RPC integration.
- `frontend/src/components/ProductGrid.tsx` — Draft receipt display and editing.
- `model/train.py` — Model training script with configuration.

15. Appendix E: Abbreviations and Standards

All monetary values in the prototype are denominated in Ethiopian Birr (ETB). Weight measurements use SI units (kilograms, grams). Detection confidence scores are dimensionless values in the range [0, 1]. mAP values are reported as decimal fractions unless stated otherwise.